



Università degli Studi di Salerno

Dipartimento di Ingegneria dell'Informazione ed Elettrica e Matematica
Applicata(DIEM)

Documentazione del Progetto

Sviluppo di un'applicazione Java per la gestione di una biblioteca

Corso di Ingegneria del Software – A.A. 2025/2026

Studenti Gruppo 11:

Vincenzo Galdiero

Matricola:0612709367

Gabriele Ciccone

Matricola :0612709695

Gerardo Carino

Matricola:0612710164

Donato Finiello

Matricola:0612709377



Indice

1. Introduzione

1.1 Scopo del documento.....	4
1.2 Contesto applicativo.....	4
1.3 Glossario.....	4

2. Progettazione ad alto livello

2.1 Obiettivi architetturali.....	5
2.2 Criteri di progettazione e Quality Attributes (QA).....	5
2.2.1 Attributi di qualità esterni.....	5
2.2.2 Attributi di qualità interni.....	6
2.3 Scomposizione object-oriented.....	6
2.4 Adozione del pattern MVC.....	7
2.5 Coesione e accoppiamento.....	7
2.6 Descrizione dei moduli.....	8
2.6.1 Modulo View.....	9
2.6.2 Modulo Gestione Libri.....	9
2.6.3 Modulo Gestione Utenti.....	10
2.6.4 Modulo Gestione Prestiti.....	10
2.6.5 Modulo Gestione Eccezioni.....	11
2.7 Viste architetturali del sistema.....	11
2.7.1 Vista logica.....	11
2.7.2 Vista di processo.....	11
2.7.3 Vista fisica.....	11
2.7.4 Vista di sviluppo.....	12
2.8 L'interfaccia Archivio come decisione architetturale.....	12
2.9 Diagramma dei package.....	13

3. Progettazione di dettaglio

3.1 Modulo Gestione Libri.....	15
3.1.1 Classe Libro.....	15
3.1.2 ArchivioLibri.....	15
3.1.3 LibriService.....	1
3.1.4 LibroViewController.....	18
3.2 Modulo Gestione Utenti.....	18
3.2.1 Classe Utente.....	18
3.2.2 ArchivioUtenti.....	18
3.2.3 UtentiService.....	18
3.2.4 UtentiViewController.....	18
3.3 Modulo Gestione Prestiti.....	19
3.3.1 Classe Prestito.....	19
3.3.2 StatoPrestiti.....	19
3.3.3 ArchivioPrestitiAttivi.....	19
3.3.4 ArchivioCronologiaPrestiti.....	19

3.3.5 PrestitiService.....	19
3.3.6 PrestitiViewController.....	19
3.4 Modulo GestioneHome.....	20
3.4.1 HomeController.....	20
3.5 Modulo GestioneEccezioni.....	20
3.6 Applicazione dei principi SOLID.....	20
3.7 Qualità del software e contenimento del debito tecnico.....	20

4. Modello statico

4.1 Diagramma delle classi - modello essenziale.....	21
4.2 Diagramma delle classi - modello completo.....	23
4.3 Descrizione dettagliata delle relazioni.....	24
4.3.1 Associazioni.....	24
4.3.2 Dipendenze.....	25
4.3.3 Aggregazioni.....	25
4.4 Analisi della coesione e dell'accoppiamento.....	25
4.4.1 Coesione.....	25
4.4.2 Accoppiamento.....	26

5. Modello dinamico

5.1 Introduzione ai diagrammi di sequenza.....	27
5.2 Sequenza: Registrazione di un nuovo libro.....	27
5.2.1 Flusso Principale.....	28
5.2.2 Gestione dei Flussi Alternativi.....	29
5.3 Sequenza: Registrazione di un prestito.....	29
5.3.1 Flusso Principale.....	29
5.3.2 Gestione dei Flussi Alternativi.....	32
5.4 Sequenza: Restituzione di un prestito.....	33
5.4.1 Flusso Principale.....	33
5.4.2 Gestione dei Flussi Alternativi.....	35
5.5 Diagramma Macchina a Stati Finiti.....	35
5.6 Descrizione del flusso dell'applicazione.....	37

6. Design dell'interfaccia utente

6.1 Linee guida adottate.....	34
6.2 Navigazione tra le schermate.....	35
6.3 Mock-up delle schermate.....	35
6.3.1 Home.....	35
6.3.2 Schermata Libri.....	36
6.3.3 Schermata Utenti.....	37
6.3.4 Schermata Prestiti.....	38
6.4 Gestione dei messaggi.....	39

1. Introduzione

1.1 Scopo del documento

Il presente documento ha lo scopo di descrivere in maniera organica la progettazione dell'applicazione Java sviluppata per la gestione di una biblioteca universitaria.

Vengono presentate:

- le scelte architetture adottate,
- la suddivisione in moduli,
- il modello statico (classi e relazioni),
- il modello dinamico (sequenze e attività),
- il design dell'interfaccia utente.

1.2 Contesto applicativo

L'applicazione è pensata per supportare le principali attività di una biblioteca universitaria:

- registrazione e gestione del catalogo dei libri;
- registrazione e gestione degli utenti;
- apertura e chiusura dei prestiti;
- rispetto dei vincoli di dominio (numero massimo di prestiti, copie disponibili, ecc.);
- persistenza dei dati tramite file.

La soluzione è stata realizzata utilizzando Java per la logica applicativa e JavaFX per l'interfaccia grafica.

1.3 Glossario

- Dominio: insieme dei concetti che descrivono biblioteca, utenti e prestiti.
- Archivio: componente che gestisce collezioni di oggetti e la loro persistenza su file.
- Service: strato che contiene le regole di business e la logica applicativa.
- Controller: classe che gestisce gli eventi dell'interfaccia utente.
- View: interfaccia grafica realizzata in FXML (JavaFX).

2. Progettazione ad alto livello

2.1 Obiettivi architetturali

Per il sistema di gestione della biblioteca è stata adottata un'architettura basata su Java e JavaFX, organizzata secondo il pattern Model–View–Controller (MVC).

Gli obiettivi principali della progettazione architetturale sono stati:

- garantire una netta separazione delle responsabilità;
- ridurre l'accoppiamento tra componenti;
- modellare il dominio in modo fedele e coerente (libri, utenti, prestiti, stati del prestito);
- avere una struttura chiara che faciliti la lettura del codice e l'individuazione dei ruoli di ogni classe.

2.2 Criteri di progettazione e Quality Attributes (QA)

La progettazione del sistema non si limita a soddisfare i requisiti funzionali, ma tiene conto anche delle caratteristiche di qualità che influenzano il comportamento complessivo dell'applicazione.

Nella fase architetturale è stato quindi necessario valutare quali attributi fossero più rilevanti per un sistema di gestione bibliotecaria.

L'obiettivo principale è stato individuare un equilibrio tra facilità d'uso, correttezza del funzionamento e capacità del sistema di adattarsi a modifiche future. Le scelte fatte nella scomposizione dei moduli, nell'organizzazione delle classi e nella definizione delle interfacce derivano direttamente da tali attributi di qualità.

2.2.1 Attributi di qualità esterni

Gli attributi di qualità esterni riguardano le caratteristiche osservabili da chi utilizza il sistema. Nel caso dell'applicazione sviluppata, sono stati considerati prioritari:

- Usabilità

L'interfaccia è stata progettata in modo semplice e intuitivo, così da permettere al bibliotecario di svolgere rapidamente le operazioni più comuni.

L'uso di JavaFX e di schermate strutturate a tabella facilita la consultazione dei dati e riduce la possibilità di errore.

- Efficienza

Le operazioni principali, come la ricerca e la visualizzazione dei cataloghi, devono essere rapide. La scelta di strutture dati appropriate contribuisce a mantenere i tempi di risposta costanti.

- Sicurezza dei dati

Pur trattandosi di un'applicazione desktop, sono state introdotte regole di validazione che impediscono l'inserimento di dati incoerenti o incompleti, proteggendo così l'integrità del catalogo e delle registrazioni degli utenti.

2.2.2 Attributi di qualità interni

Gli attributi interni riguardano la struttura del codice e la sua organizzazione

- Manutenibilità

Il sistema è suddiviso in moduli fortemente coesi, ciascuno dedicato a un insieme limitato e ben definito di responsabilità. Questo consente di localizzare facilmente le parti del codice da modificare e di intervenire senza rischiare effetti collaterali sugli altri moduli.

- Modularità

La separazione tra entità di dominio, archivi, servizi e controllori riduce l'accoppiamento e permette di ampliare o sostituire una componente senza impattare sull'intero progetto.

- Riusabilità

L'architettura adottata, basata su dominio, archivi, service e controller, può essere riutilizzata integralmente per sistemi analoghi, ad esempio per gestire il noleggio di dispositivi o materiali multimediali, cambiando solo le entità del dominio.

- Testabilità

La chiara separazione tra logica applicativa e interfaccia grafica rende possibile testare i servizi e gli archivi in modo isolato.

- Portabilità

La scelta di Java permette di eseguire l'applicazione su sistemi diversi senza modifiche, caratteristica utile nel contesto universitario.

2.3 Scomposizione object-oriented

La progettazione del sistema è stata guidata dall'obiettivo di ottenere una struttura chiara e coerente con il dominio applicativo della biblioteca. Tra i diversi approcci possibili, è stato adottato un modello di progettazione object-oriented, ritenuto il più adeguato per rappresentare entità reali come libri, utenti e prestiti.

In fase preliminare sono stati valutati i due approcci canonici alla scomposizione del sistema:

- scomposizione funzionale, che frammenta il software in procedure e operazioni elementari da eseguire in sequenza;
- scomposizione orientata agli oggetti, che organizza il sistema attorno a entità dotate sia di dati sia di comportamenti.

La prima modalità risulta efficace per sistemi puramente procedurali, ma tende a generare strutture difficili da estendere e con bassa coesione. Nel contesto della gestione bibliotecaria, al contrario, il problema è naturalmente modellabile attraverso oggetti che hanno un'identità e uno stato: ad esempio un libro con il suo ISBN, un utente con la sua matricola o un prestito con il proprio ciclo di vita.

Per questo motivo l'approccio orientato agli oggetti si è rivelato più adatto: consente di associare i dati e le operazioni che li riguardano all'interno della stessa classe, migliorando la chiarezza dei ruoli e la coesione interna dei moduli. Inoltre, la progettazione a oggetti facilita il controllo dell'accoppiamento, poiché le interazioni fra le componenti avvengono attraverso metodi e interfacce ben definite, evitando dipendenze non necessarie.

2.4 Adozione del pattern MVC

Nella progettazione del sistema è stato adottato il pattern Model–View–Controller (MVC), una soluzione architetturale che consente di separare in modo netto i dati, la logica applicativa e l'interfaccia utente. Tale scelta risponde direttamente alle esigenze di qualità identificate nella fase architetturale.

L'adozione del pattern nasce dalla necessità di evitare che la gestione grafica e la logica del dominio si influenzino reciprocamente. In un sistema come quello della biblioteca, infatti, le operazioni legate al catalogo, agli utenti e ai prestiti devono poter evolvere nel tempo senza richiedere modifiche all'interfaccia grafica, e viceversa. La separazione proposta da MVC permette di ottenere questo risultato attraverso tre componenti con ruoli distinti:

- **Model:** rappresenta lo strato più vicino al dominio applicativo. Include le entità (Libro, Utente, Prestito), gli archivi per la gestione dei dati e servizi. Lo stato del sistema e i vincoli di dominio risiedono interamente in questa parte.
- **View:** comprende le interfacce grafiche realizzate con JavaFX e definite tramite file FXML. Le view mostrano all'utente l'informazione elaborata dal sistema, ma non applicano alcuna logica né manipolano direttamente i dati.
- **Controller:** funge da ponte tra vista e modello. Intercetta gli input provenienti dall'interfaccia grafica, richiama i servizi appropriati e aggiorna la view in base allo stato corrente del sistema.

2.5 Coesione e accoppiamento

La qualità strutturale del sistema dipende dal modo in cui le sue parti sono state suddivise e collegate tra loro. Per progettare un software facilmente modificabile e stabile nel tempo, sono stati seguiti due principi fondamentali: alta coesione all'interno dei moduli e basso accoppiamento tra componenti diversi.

Coesione

La coesione indica quanto le responsabilità presenti in un modulo siano legate tra loro.

Nel progetto si è puntato a moduli coesi, ciascuno dedicato a un compito specifico del dominio:

- il modulo dei libri gestisce il catalogo e le sue regole;
- quello degli utenti si occupa dell'anagrafica e dei controlli associati;
- il modulo prestiti governa il ciclo di vita dei prestiti;
- le eccezioni sono riunite in un package unico.

Questa organizzazione rende ogni parte autonoma e facilmente comprensibile e rispecchia una coesione funzionale, il livello più elevato, perché ogni modulo realizza un unico compito ben definito.

Accoppiamento

L'accoppiamento misura quanto un modulo dipende da un altro.

Per ridurre interdipendenze inutili, la comunicazione tra componenti avviene solo attraverso metodi pubblici e non tramite accesso diretto alle strutture interne.

Gli archivi incapsulano i propri dati, i servizi mediano tra controller e modello, e l'interfaccia Archivio limita ulteriormente le dipendenze tra le varie parti.

Il risultato è un sistema che utilizza un accoppiamento per dati, uno dei livelli più desiderabili, evitando casi come variabili globali o strutture condivise.

2.6 Descrizione dei moduli

Per mantenere il codice ordinato e facilmente modificabile, il sistema è stato suddiviso in più moduli indipendenti, ciascuno dedicato a un'area funzionale.

La Figura 1 riporta la vista statica dell'architettura tramite il *Diagramma dei Package*, che rappresenta il package principale biblioteca articolato nei seguenti sottopackage:

- `gestioneLibri`: contiene le classi responsabili della gestione del catalogo librario, delle validazioni e delle operazioni di archiviazione.
- `gestioneUtenti`: racchiude gli elementi dedicati all'anagrafica degli utenti e ai controlli relativi alla loro gestione.
- `gestionePrestiti`: rappresenta il cuore del sistema, occupandosi del ciclo di vita dei prestiti e dell'integrazione tra libri e utenti.
- `gestioneHome`: include i componenti della schermata iniziale e della navigazione interna.
- `gestioneEccezioni`: raccoglie tutte le eccezioni personalizzate utilizzate a livello applicativo.
- `view`: package esterno a biblioteca, dedicato esclusivamente all'interfaccia grafica realizzata in JavaFX.

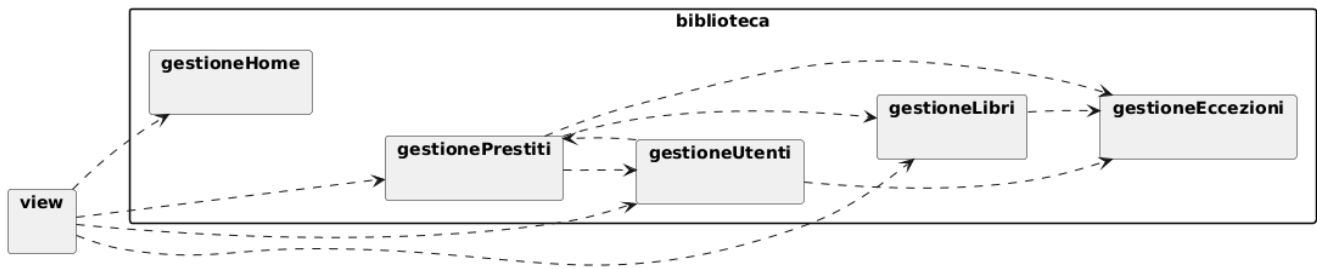


Figura 1 – Diagramma dei package dell'applicazione

2.6.1 Modulo View

Questo modulo raccoglie tutte le schermate con cui il bibliotecario interagisce durante l'uso dell'applicazione.

L'interfaccia è realizzata in JavaFX, definita tramite file FXML progettati con l'ausilio di SceneBuilder, così da separare in modo netto la parte grafica dal codice Java.

All'interno di questo package vengono gestiti:

- la visualizzazione dei dati in tabelle, etichette (label) e campi di input;
- la raccolta degli input inseriti dall'utente (click, testo, selezioni);
- l'invio di questi input ai controller corrispondenti;
- la presentazione di messaggi informativi, di errore o di conferma.

In questo modo la view si occupa esclusivamente di "come" mostrare le informazioni, lasciando tutta la logica di calcolo e controllo agli altri livelli dell'applicazione.

2.6.2 Modulo GestioneLibri

Il modulo di GestioneLibri ha il compito di amministrare il catalogo della biblioteca.

Non si limita a memorizzare o cancellare i libri, ma svolge un vero e proprio ruolo di controllo e tutela dell'integrità dei dati.

Quando viene inserito un nuovo libro, il modulo verifica che:

- i dati inseriti siano completi e coerenti;
- non esistano duplicati rispetto ai libri già presenti.

Allo stesso modo, prima di rimuovere un libro dal catalogo, il sistema controlla che non sia ancora legato a prestiti attivi, evitando così situazioni incoerenti.

All'interno del modulo sono presenti:

- la classe Libro, che rappresenta il singolo volume;

- ArchivioLibri, che mantiene la collezione dei libri gestiti dal sistema;
- LibriService, che applica le regole di dominio (validazioni, controlli, vincoli);
- LibroViewController, che mette in comunicazione la view con il servizio.

2.6.3 Modulo GestioneUtenti

Questo modulo si occupa della gestione degli utenti iscritti alla biblioteca, curandone l'anagrafica e il rispetto delle regole a loro collegate. Il funzionamento è analogo a quello del modulo libri, ma applicato alle persone registrate nel sistema.

In particolare, il modulo:

- controlla che la matricola rispetti il formato previsto ed sia univoca;
- verifica che tutti i campi obbligatori (nome, cognome, e-mail, ecc.) siano compilati;
- impedisce l'eliminazione di un utente che risulta ancora titolare di uno o più prestiti attivi.

Le classi principali del modulo sono:

- Utente, che rappresenta il singolo iscritto;
- ArchivioUtenti, che conserva e gestisce l'insieme degli utenti;
- UtentiService, che si occupa di validazione, inserimenti, modifiche e cancellazioni;
- UtentiViewController, che mette in comunicazione la view per realizzare l'interfaccia grafica dedicata alla gestione utenti.

2.6.4 Modulo GestionePrestiti

Il modulo di Gestione Prestiti rappresenta il "cuore" funzionale dell'applicazione, perché mette in relazione i libri con gli utenti e segue l'intero ciclo di vita del prestito.

A partire dalla richiesta iniziale fino alla restituzione, il modulo:

- verifica se l'utente può aprire un nuovo prestito, controllando il limite massimo di tre prestiti attivi;
- controlla che il libro richiesto abbia ancora copie disponibili;
- aggiorna automaticamente il numero di copie disponibili quando un prestito viene aperto o chiuso;
- registra e aggiorna lo stato del prestito (ad esempio: attivo, in ritardo, chiuso), gestendo anche eventuali ritardi nella restituzione.

Le principali componenti del modulo sono:

- Prestito e l'enumerazione StatoPrestiti, che descrivono il singolo prestito e il suo stato;
- ArchivioPrestitiAttivi e ArchivioCronologiaPrestiti, che distinguono i prestiti correnti da quelli conclusi e attivi;
- PrestitiService, che implementa tutte le regole e i controlli specifici di questa parte del dominio;

- `PrestitiViewController`, che con la sua view associata, permette al bibliotecario di visualizzare, aprire e chiudere i prestiti.

2.6.5 Modulo GestioneEccezioni

Il modulo dedicato alla gestione delle eccezioni racchiude tutti gli errori specifici che possono verificarsi durante l'utilizzo dell'applicazione.

La presenza di un package separato permette di mantenere ordinato il codice dei servizi e garantire messaggi d'errore chiari e coerenti in ogni parte del sistema.

2.7 Viste architettureali del sistema

Il progetto della biblioteca può essere descritto efficacemente attraverso le quattro viste principali del modello di *Kruchten*.

2.7.1 Vista logica

La vista logica rappresenta la struttura concettuale del sistema e mostra come le principali funzionalità sono raggruppate in moduli coerenti. Nel nostro progetto, il dominio è articolato attorno a tre nuclei informativi, Libri, Utenti e Prestiti, ciascuno modellato attraverso classi dedicate (entità, archivi e servizi) e supportato da un package specifico.

2.7.2 Vista di processo

La vista di processo mostra come i componenti principali interagiscono durante l'esecuzione del sistema. Il flusso tipico segue la catena:

View → Controller → Service → Archivio → Model

- La View raccoglie gli input dell'utente.
- Il Controller gestisce gli eventi e chiama i servizi.
- I Service applicano le regole di dominio (prestiti, validazioni, controlli).
- Gli Archivi gestiscono i cataloghi la persistenza tramite file.
- Le entità del Model aggiornano il loro stato.

Questa vista è ben rappresentata dai diagrammi di sequenza (presentati più avanti), che mostrano in dettaglio la collaborazione tra componenti nelle operazioni principali.

2.7.3 Vista fisica

Questa vista descrive come il software è mappato sulle risorse fisiche. Poiché l'applicazione è desktop, l'allocazione è semplice e lineare.

Il codice Java gira sulla JVM locale, JavaFX gestisce la parte grafica sulla stessa macchina e i file usati come archivi vengono salvati nel file system locale.

2.7.4 Vista di sviluppo

La vista di sviluppo descrive come il progetto è organizzato dal punto di vista degli sviluppatori, cioè come è strutturato il codice sorgente e come i componenti sono distribuiti all'interno dei package.

Nel nostro sistema, la decomposizione segue una logica object-oriented e riflette fedelmente la suddivisione per responsabilità definita in fase architetturale. Il codice è organizzato in cinque macro-package principali, ai quali si aggiunge il package *view* dedicato all'interfaccia grafica JavaFX.

Questa struttura supporta il lavoro collaborativo del team, permette una chiara distribuzione dei compiti.

2.8 L'interfaccia Archivio come decisione architetturale

L'interfaccia Archivio rappresenta una scelta architetturale strategica volta a ridurre l'accoppiamento. Essa definisce un contratto comune che tutti gli archivi devono rispettare:

- leggiDaFile(String filename)
- scriviSuFile(String filename)

Questa astrazione è presentata in Figura 2 e presenta tre vantaggi principali:

1. Uniformità: tutti gli archivi (Libri, Utenti, Prestiti Attivi, Cronologia) rispettano lo stesso schema operativo.
2. Riduzione dell'accoppiamento: i service non dipendono dall'implementazione concreta, ma solo dal comportamento previsto dall'interfaccia.
3. Estendibilità: per aggiungere un nuovo archivio basterà implementare l'interfaccia, senza modificare il resto del sistema.

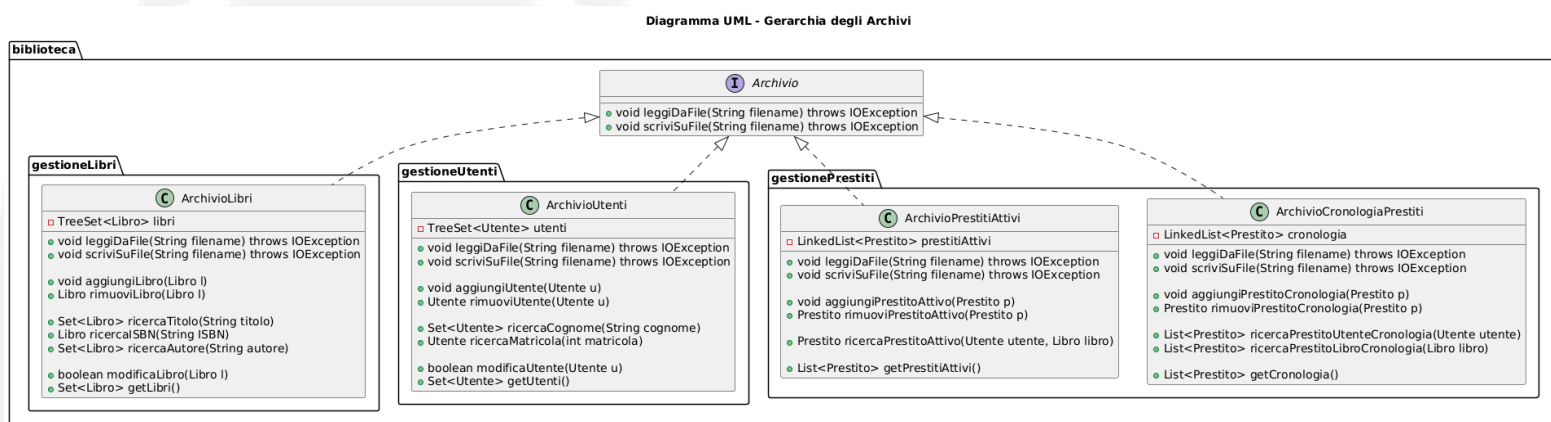


Figura 2- Diagramma UML sulla gerarchia degli archivi

2.9 Diagramma dei package

Il diagramma dei package (Figura 3) rappresenta la suddivisione complessiva del progetto nei suoi moduli principali.

Il package radice *biblioteca* raccoglie i cinque moduli che costituiscono il cuore dell'applicazione:

- *gestioneLibri*: comprende le classi dedicate al dominio dei libri, alla loro persistenza e ai servizi che ne regolano il comportamento;
- *gestioneUtenti*: contiene le classi relative all'anagrafica, ai servizi dedicati e all'archivio degli utenti registrati;
- *gestionePrestiti*: include il modello dei prestiti, gli archivi (attivi e storico), la logica applicativa e il controller grafico associato;
- *gestioneHome*: gestisce la schermata iniziale e la navigazione verso le diverse sezioni dell'applicazione;
- *gestioneEccezioni*: raccoglie tutte le eccezioni specializzate utilizzate nei vari moduli.

Al di fuori di *biblioteca* è presente il package *view*, dedicato esclusivamente ai file FXML e alla parte grafica realizzata in JavaFX.

Il diagramma evidenzia anche le dipendenze principali tra i package: *gestioneLibri*, *gestioneUtenti* e *gestionePrestiti* dipendono dal package *gestioneEccezioni*, mentre il package *view* fa riferimento a tutti i package di gestione per interagire con i rispettivi controller. Inoltre, *gestionePrestiti* comunica sia con *gestioneLibri* sia con *gestioneUtenti* per le operazioni che coinvolgono libri e utenti, e il package *gestioneUtenti* mantiene a sua volta una dipendenza da *gestionePrestiti* per la consultazione dei prestiti associati agli utenti.



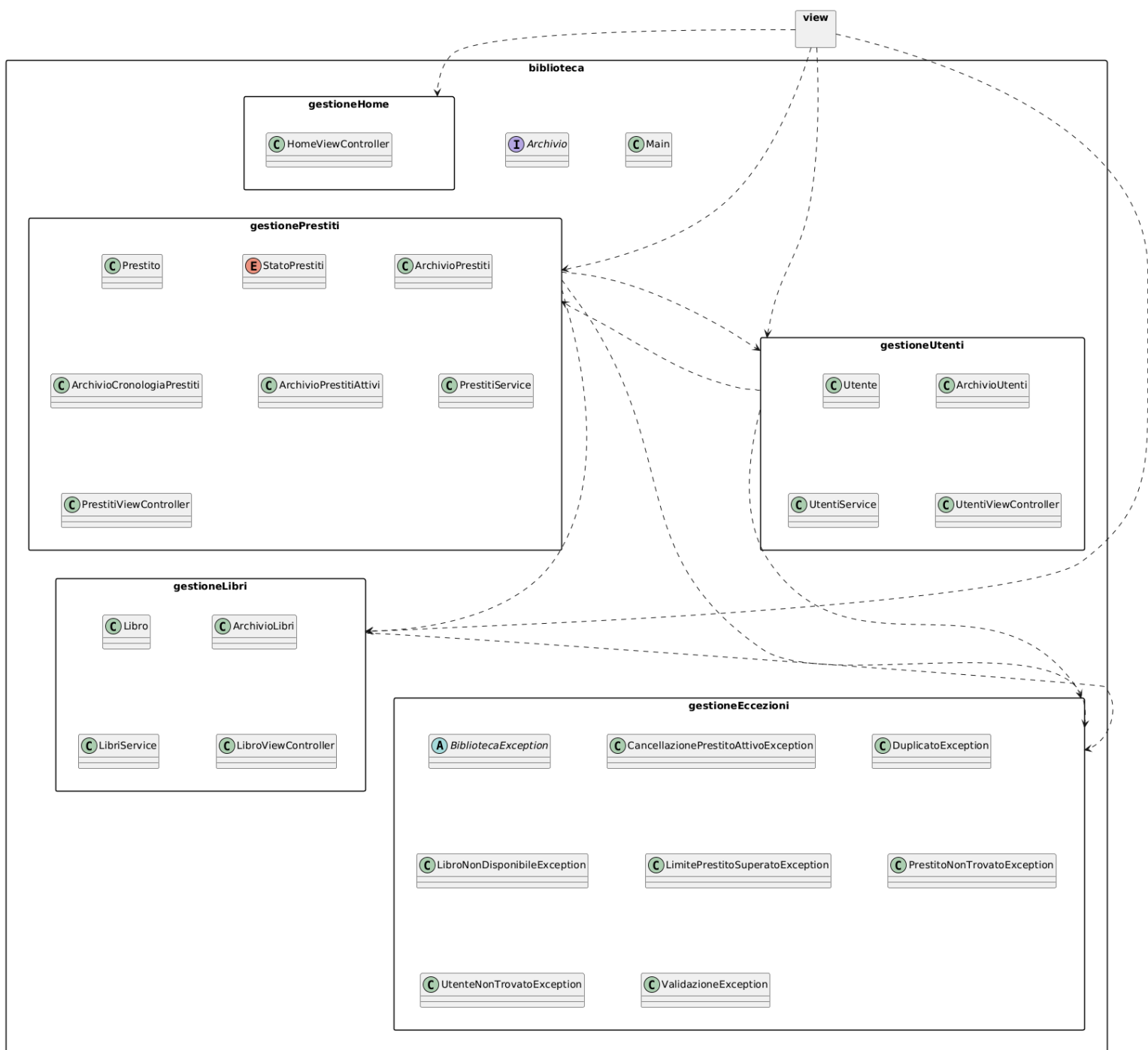


Figura 3 - Diagramma dei package

3. Progettazione di dettaglio

In questa sezione scendiamo nel "cuore" del sistema, analizzando come abbiamo tradotto l'architettura generale in classi concrete.

3.1 Modulo Gestione Libri

Il pacchetto `gestioneLibri` è stato isolato per contenere tutto ciò che riguarda il ciclo di vita dei volumi.

3.1.1 Classe Libro

La classe contiene i dati anagrafici e la gestione del numero di copie. Inoltre implementa l'interfaccia per la comparazione (`Comparable`) e sovrascrive `equals`, `hashCode` e `toString`: questo è fondamentale perché ci permette di inserire i libri in collezioni ordinate (come `TreeSet`), di identificare univocamente un libro tramite il suo ISBN, evitando duplicati logici e di stampare gli attributi di libro.

3.1.2 ArchivioLibri

È il componente responsabile della persistenza dei dati relativi ai libri. Implementando l'interfaccia comune *Archivio*, la classe gestisce la lettura e la scrittura su file dell'intera collezione, garantendo che i dati vengano ripristinati correttamente all'avvio del sistema. Al suo interno mantiene la struttura dati che raccoglie tutti i libri e mette a disposizione soltanto operazioni controllate per l'aggiunta, la rimozione, la modifica, la visualizzazione e la ricerca.

3.1.3 LibriService

Il `LibriService` agisce da filtro tra l'utente e i dati: prima che un libro raggiunga l'archivio, il servizio controlla che i dati siano validi (es. copie non negative) e che non esistano conflitti (ISBN già presente). Inoltre blocca la cancellazione di un libro se il sistema rileva che è attualmente in prestito.

3.1.4 LibroViewController

Il Controller gestisce l'interazione tra l'utente e il sistema. Non contiene logica applicativa, ma si limita a trasformare le azioni del bibliotecario in richieste al Service e ad aggiornare l'interfaccia con i dati ottenuti. Inoltre intercetta le eccezioni provenienti dai livelli inferiori e le traduce in messaggi di errore chiari e comprensibili.

3.2 Modulo GestioneUtenti

Analogamente al modulo libri, *gestioneUtenti* si occupa della gestione delle persone registrate nel sistema. L'aspetto principale consiste nel garantire la coerenza tra i dati dell'utente e i prestiti attualmente associati.

3.2.1 Classe Utente

L'entità *Utente* non è soltanto un contenitore dei dati anagrafici, ma gestisce anche l'elenco dei prestiti attivi associati alla persona. Questa struttura interna permette di controllare rapidamente se l'utente ha raggiunto il limite massimo di prestiti consentiti, senza dover scansionare l'intero archivio. Inoltre, la classe sovrascrive i metodi *equals*, *hashCode* e *toString*, così da garantire un'identificazione univoca dell'utente tramite la sua matricola, evitare duplicati logici nelle collezioni e consentire una stampa leggibile dei suoi dati.

3.2.2 ArchivioUtenti

È il componente responsabile della persistenza dei dati relativi agli utenti. Implementando l'interfaccia comune *Archivio*, la classe gestisce la lettura e la scrittura su file dell'intera collezione, assicurando che gli iscritti vengano caricati correttamente all'avvio del sistema. Al suo interno mantiene la struttura dati che raccoglie tutti gli utenti in modo ordinato e mette a disposizione soltanto operazioni controllate per l'aggiunta, la rimozione, la modifica, la visualizzazione e la ricerca, garantendo anche l'unicità della matricola per evitare duplicati logici.

3.2.3 UtentiService

L'*UtentiService* funge da filtro tra l'interfaccia e i dati relativi agli utenti: prima che un nuovo utente venga registrato, il servizio verifica la correttezza formale delle informazioni (ad esempio il formato dell'email istituzionale) e controlla l'assenza di conflitti, come la duplicazione della matricola. Inoltre impedisce la rimozione di un utente se risultano ancora prestiti attivi a suo nome, evitando così inconsistenze e garantendo che nessun volume venga "perso" dal sistema.

3.2.4 UtentiViewController

Il controller gestisce l'interazione tra l'operatore e l'anagrafica degli utenti. Non contiene logica applicativa, ma traduce le azioni dell'utente in richieste allo *UtentiService* e aggiorna l'interfaccia mostrando i dati correnti nella tabella. Inoltre si occupa di intercettare eventuali eccezioni, trasformandole in messaggi di feedback chiari e comprensibili.

3.3 Modulo Gestione Prestiti

È il modulo più complesso, perché svolge il ruolo di collegamento tra la gestione dei libri e quella degli utenti. Controlla l'evoluzione temporale dei prestiti — dalla creazione alla restituzione — coordinando le verifiche sulle copie disponibili e sul numero di libri già in carico all'utente.

3.3.1 Classe Prestito

La classe *Prestito* rappresenta il legame tra un utente e un libro. Oltre ai riferimenti alle due entità, registra le principali date operative del prestito — inizio, scadenza prevista e

restituzione effettiva. È l'oggetto che passa dall'archivio dei prestiti attivi alla cronologia una volta conclusa l'operazione.

3.3.2 StatoPrestiti

Per evitare l'utilizzo di stringhe o codici numerici poco significativi, lo stato del prestito è modellato tramite un'enumerazione che include i valori ATTIVO, RESTITUITO e RITARDO. Questa scelta migliora la leggibilità del codice e garantisce che a un prestito possano essere assegnati solo stati validi.

3.3.3 ArchivioPrestitiAttivi

È il componente dedicato alla gestione e alla persistenza dei prestiti attualmente in corso. Implementando l'interfaccia comune *Archivio*, la classe si occupa della lettura e scrittura su file dell'insieme dei prestiti attivi, garantendo che lo stato dell'applicazione venga ripristinato correttamente all'avvio. Al suo interno mantiene la struttura dati (*LinkedList*) che raccoglie solo le operazioni di prestito ancora aperte e mette a disposizione metodi controllati per l'aggiunta, la rimozione, la visualizzazione e la ricerca.

3.3.4 ArchivioCronologiaPrestiti

È il componente dedicato alla conservazione e alla persistenza dei prestiti. Come gli altri archivi, implementa l'interfaccia comune *Archivio* e gestisce la lettura e la scrittura su file dell'intera cronologia, assicurando che lo storico venga mantenuto in modo completo e coerente nel tempo tramite struttura dati *LinkedList*. Internamente raccoglie tutti i prestiti chiusi, attivi e in ritardo offrendo operazioni controllate per la consultazione e la ricerca, senza permettere modifiche improprie ai dati storici.

L'*ArchivioCronologia* garantisce una tracciabilità completa e ordinata dell'intera storia di un utente o di un libro, registrando ogni prestito concluso in modo permanente. In questo modo è possibile ricostruire con precisione tutte le operazioni avvenute nel tempo, analizzare i comportamenti degli utenti, verificare l'utilizzo dei libri e supportare eventuali controlli o statistiche, senza interferire con la gestione dei prestiti attivi.

3.3.5 PrestitiService

Il *PrestitiService* costituisce il punto di coordinamento centrale per tutte le operazioni relative ai prestiti. Ogni richiesta viene validata applicando una serie di controlli incrociati: il servizio verifica innanzitutto che il libro richiesto sia effettivamente disponibile e che esistano copie prestabili; successivamente controlla che l'utente non abbia raggiunto il numero massimo di prestiti consentiti o eventuali condizioni che ne impediscano l'erogazione.

Solo quando tutte le verifiche risultano positive, il servizio procede alla creazione dell'oggetto *Prestito*, aggiorna gli archivi interessati e decrementa il numero di copie disponibili del libro.

3.3.6 PrestitiViewController

Il controller gestisce l'interazione dell'operatore con l'interfaccia dei prestiti. Non contiene logica applicativa, ma si limita a trasformare le azioni dell'utente — come la selezione di un libro o di un utente, la registrazione di un nuovo prestito o la restituzione di uno già esistente — in richieste al *PrestitiService*. Inoltre aggiorna la vista mostrando lo stato corrente delle operazioni, inclusi eventuali ritardi, e intercetta gli errori provenienti dai livelli inferiori, presentandoli attraverso messaggi chiari e comprensibili.

3.4 Modulo GestioneHome

Il modulo *gestioneHome* fornisce il punto di accesso principale all'applicazione e gestisce la navigazione tra le diverse sezioni del sistema. Offre un'interfaccia semplice che permette all'utente di raggiungere rapidamente i moduli dedicati a libri, utenti e prestiti.

3.4.1 HomeViewController

Il controller funge da punto di accesso principale all'applicazione e gestisce la navigazione tra le diverse sezioni. Non contiene logica applicativa, ma si occupa di inizializzare il layout principale e di instradare l'utente verso le varie viste, garantendo un passaggio fluido e coerente tra i moduli della biblioteca.

3.5 Modulo GestioneEccezioni

Per evitare di distribuire la gestione degli errori in modo disordinato all'interno del codice, è stato introdotto un pacchetto dedicato alle eccezioni personalizzate. Questo modulo centralizza tutte le condizioni di errore legate al dominio, rendendo il codice più pulito, leggibile e facile da mantenere.

Alla base della gerarchia troviamo *BibliotecaException*, la classe astratta da cui derivano tutte le altre eccezioni definite dal progetto.

Ogni eccezione figlia rappresenta un caso ben preciso, facilitando la gestione delle anomalie e permettendo ai controller di comunicare all'utente messaggi mirati.

Le principali eccezioni presenti sono:

- *DuplicatoException*: viene lanciata quando si tenta di inserire un elemento già presente nell'archivio, come un libro con ISBN già esistente o un utente con matricola duplicata.
- *ValidazioneException*: segnala errori nei dati inseriti, come campi mancanti, formati non validi o valori incoerenti.
- *LibroNonDisponibileException*: indica che il libro richiesto non ha copie disponibili per il prestito.
- *LibroNonTrovatoException*: viene sollevata quando il libro cercato non è presente nel catalogo.
- *UtenteNonTrovatoException*: indica che non esiste nessun utente associato alla matricola o ai criteri inseriti.

- `PrestitoNonTrovatoException`: si verifica quando non è presente un prestito attivo o registrato per l'utente o il libro selezionato.
- `LimitePrestitoSuperatoException`: segnala che l'utente ha già raggiunto il numero massimo di prestiti attivi consentiti.
- `CancellazionePrestitoAttivoException`: impedisce la rimozione di entità coinvolte in prestiti ancora attivi, come la cancellazione di un utente che non ha restituito tutti i libri.

3.6 Applicazione dei principi SOLID

L'architettura del sistema non è frutto di scelte casuali, ma segue in modo consapevole i principi SOLID, così da garantire qualità, estensibilità e affidabilità.

S — Single Responsibility Principle

Ogni componente del sistema ha una responsabilità chiara e unica:

- le *classi di dominio* rappresentano i dati;
 - gli *Archivi* gestiscono la persistenza e la memorizzazione delle raccolte;
 - i *Service* applicano le regole e coordinano le operazioni;
 - i *Controller* mediano l'interazione con l'interfaccia grafica.
- Questa suddivisione evita sovrapposizioni e rende il codice più semplice da comprendere e mantenere.

O — Open/Closed Principle

Il sistema è progettato per essere aperto all'estensione e chiuso alle modifiche.

L'aggiunta di nuove entità (ad esempio "Riviste" o "Dispositivi multimediali") richiederebbe solo di definire nuove classi di dominio e nuovi archivi specifici, senza alterare la logica esistente. La struttura modulare permette quindi di far crescere il software senza introdurre regressioni o modifiche invasive.

L — Liskov Substitution Principle / I — Interface Segregation / D — Dependency Inversion

Questi tre principi vengono applicati attraverso l'introduzione dell'interfaccia `Archivio`, che fornisce un contratto comune per tutte le componenti dedicate alla persistenza dei dati. Ogni archivio — libri, utenti, prestiti attivi e cronologia — implementa la stessa interfaccia mantenendo però una propria struttura interna e modalità operative. Questa soluzione garantisce:

- una **struttura coerente** tra tutti gli archivi;
- la **possibilità di sostituire o estendere** un archivio senza modificare la logica applicativa;
- una **minore dipendenza** dai dettagli concreti e maggiore facilità di manutenzione.

3.7 Qualità del software e contenimento del debito tecnico

Il progetto è stato sviluppato con l'obiettivo di mantenere basso il debito tecnico, cioè la quantità di complessità non gestita che potrebbe rendere difficili le evoluzioni future.

La divisione in pacchetti coerenti, la presenza di archivi specializzati, l'uso di eccezioni personalizzate e la chiara separazione tra dominio, service e interfaccia consentono di isolare ogni responsabilità e di individuare immediatamente il punto in cui intervenire in caso di modifica.

Ad esempio, se in futuro si decidesse di cambiare il numero massimo di prestiti consentiti, l'intervento sarebbe limitato al *PrestitiService*, senza effetti collaterali su altri moduli.

Analogamente, la sostituzione della persistenza su file con un database comporterebbe modifiche esclusivamente negli archivi, lasciando invariata la logica di business e l'interfaccia utente.

Questo approccio riduce il rischio di introdurre errori, semplifica le attività di testing e garantisce una manutenzione sostenibile nel tempo.



4. Modello statico

Il modello statico descrive la struttura interna del sistema, mettendo in evidenza gli elementi fondamentali del dominio e le relazioni tra essi.

La progettazione è stata suddivisa in due livelli:

1. un diagramma essenziale, che raccoglie esclusivamente gli oggetti del dominio e gli archivi che li gestiscono;
2. un diagramma completo, che estende il primo introducendo i servizi applicativi, le eccezioni personalizzate e il layer di interfaccia grafica.

Questa scelta consente di comprendere prima la logica fondamentale descritta nello SRS (prestiti, utenti, libri, vincoli e regole) per poi contestualizzarla all'interno dell'architettura software effettiva.

4.1 Diagramma delle classi – modello essenziale

Il primo diagramma descrive la parte più fondamentale del sistema, cioè l'insieme delle classi che rappresentano gli elementi centrali del dominio applicativo e gli archivi che ne consentono la gestione.

Questa sezione costituisce la traduzione più diretta dei requisiti individuati nello SRS, perché raccoglie tutti gli aspetti indispensabili al funzionamento della biblioteca digitale.

Dunque il diagramma in Figura 4 costituisce la base concettuale dell'intero sistema: è qui che vengono definite le regole e i rapporti fondamentali tra utenti, libri e prestiti, prima di introdurre ulteriori elementi architetturali che, nel diagramma successivo, si occuperanno della logica applicativa e dell'interazione con l'interfaccia grafica.

Le classi fondamentali individuate sono:

Classi di dominio

- Libro: rappresenta un volume presente in biblioteca; contiene titolo, autore, ISBN, anno di pubblicazione e numero di copie disponibili.
- Utente: modella la persona registrata presso il sistema; include nome, cognome, matricola e indirizzo e-mail istituzionale, oltre alla lista dei prestiti attivi.
- Prestito: rappresenta l'atto del prestito di un libro, includendo data di inizio, data prevista di restituzione, eventuale data effettiva e stato del prestito.
- StatoPrestiti: enumerazione che definisce i possibili stati (ATTIVO, RITARDO, CHIUSO).

Archivi

Gli archivi permettono di organizzare e mantenere le istanze delle classi di dominio:

- ArchivioLibri, responsabile della gestione, ricerca e memorizzazione dei libri;
- ArchivioUtenti, che mantiene l'insieme degli utenti registrati;
- ArchivioPrestitiAttivi, che conserva i soli prestiti correnti;

- ArchivioCronologiaPrestiti, dove vengono inseriti i prestiti attivi e in ritardo, i prestiti chiusi.

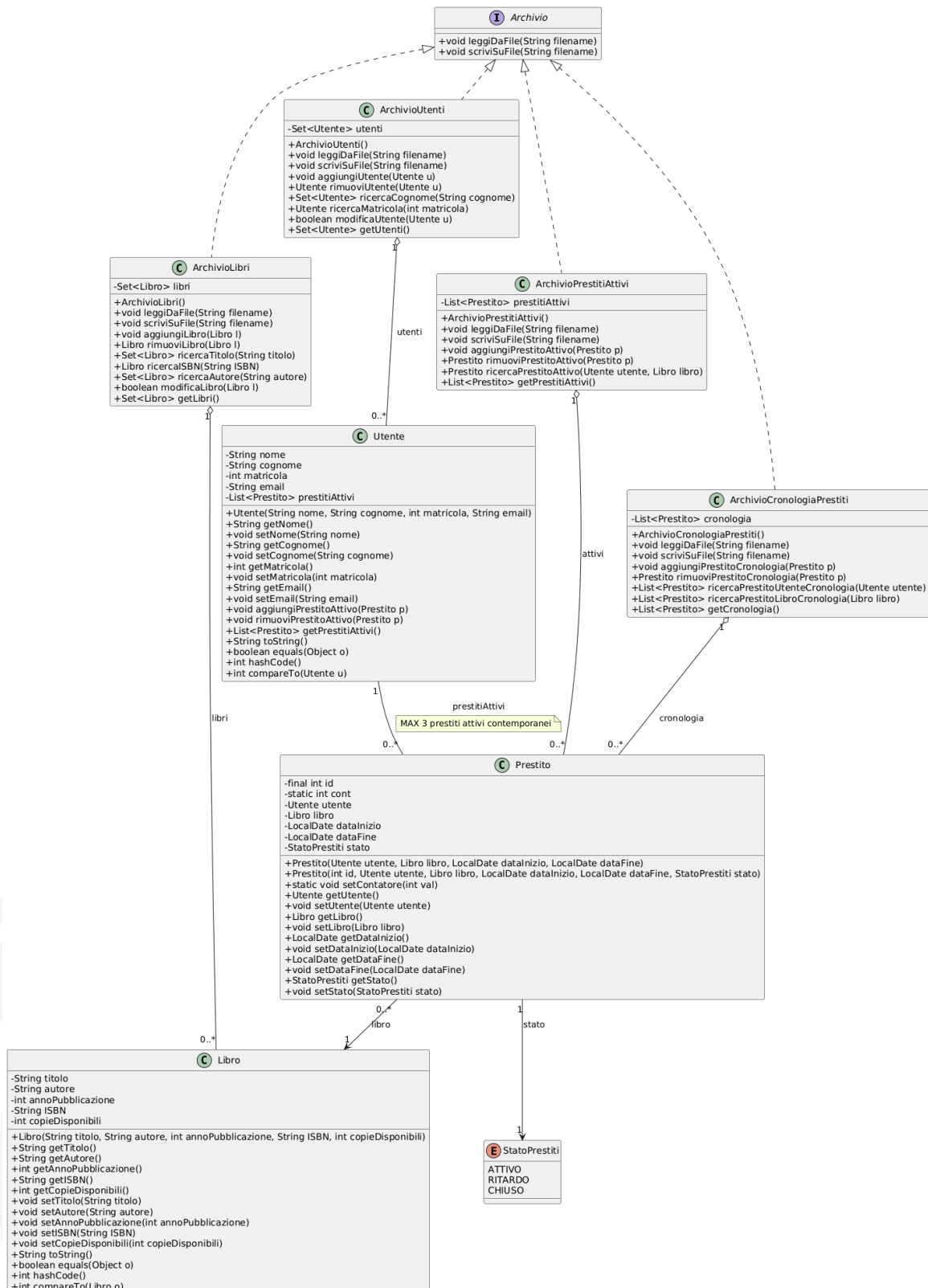


Figura 4 - Diagramma delle classi essenziale

4.2 Diagramma delle classi – modello completo

La Figura 5 rappresenta il secondo diagramma delle classi che amplia il modello con gli elementi necessari all'implementazione Java:

- Service (*LibriService*, *UtentiService*, *PrestitiService*) che incapsulano la logica applicativa;
- ViewController per la gestione dell'interfaccia JavaFX;
- Gerarchia delle eccezioni, derivate da *BibliotecaException*, che consentono una gestione controllata degli errori di dominio (duplicati, limiti superati, libro non disponibile, ecc.);
- Classe astratta Archivio, utilizzata come base per l'operazione di lettura/scrittura su file.

Questa estensione mantiene intatto il nucleo concettuale del sistema, ma consente di vedere come la struttura applicativa realizza concretamente i requisiti e come si distribuiscono le responsabilità.

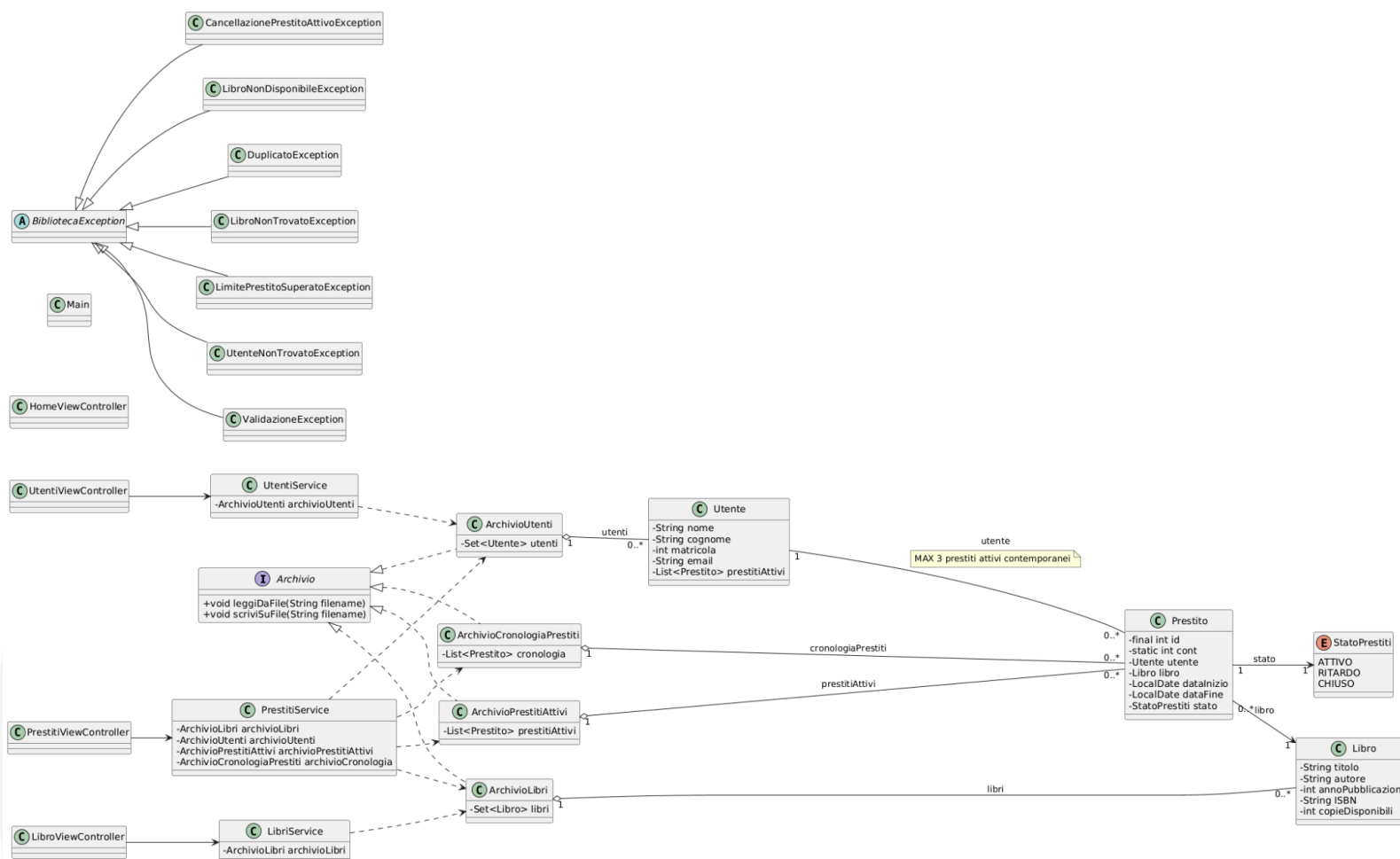


Figura 5- Diagramma delle classi completo

4.3 Descrizione dettagliata delle relazioni

4.3.1 Associazioni

Utente – Prestito (1 – 0..*)

La relazione tra *Utente* e *Prestito* è di tipo uno-a-molti, poiché ogni utente può avere più prestiti attivi contemporaneamente (nello specifico massimo 3 prestiti attivi contemporaneamente). La classe *Utente* mantiene infatti una lista di prestiti, che inizialmente è vuota al momento della registrazione. D'altra parte, ogni prestito è sempre riferito a un solo utente, in quanto rappresenta un'operazione identificata univocamente. Questa relazione è l'unica bidirezionale del modello, ed è strutturata così per ragioni funzionali:

- l'utente deve poter accedere rapidamente ai propri prestiti attivi;
- ogni prestito deve indicare con precisione l'utente a cui è stato assegnato.

Prestito – Libro (0..* – 1)

La relazione tra *Prestito* e *Libro* è una classica molti-a-uno: nel tempo un libro può essere coinvolto in numerosi prestiti, mentre ogni prestito fa riferimento a un unico volume.

La responsabilità di verificare se il libro è disponibile, e quindi se è possibile emettere un nuovo prestito, appartiene ai *Service*, che gestiscono l'aggiornamento del numero di copie disponibili e impediscono la creazione di prestiti non validi.

Prestito – StatoPrestiti (1 – 1)

Ogni prestito è associato a uno stato, rappresentato dall'enumerazione *StatoPrestiti*.

Lo stato del prestito svolge un ruolo centrale nella logica applicativa: determina se l'operazione è ancora attiva, se deve essere segnalata in ritardo oppure se può essere archiviata nella cronologia.

La relazione è unidirezionale poiché lo stato non necessita di conoscere il prestito a cui è associato.

4.3.2 Dipendenze

Service – Archivi

Le classi *service* dipendono dagli archivi per eseguire le operazioni sui dati persistenti (ricerche, salvataggi, aggiornamenti).

Si tratta di una relazione di dipendenza: i service non creano né gestiscono il ciclo di vita degli archivi, ma li utilizzano come strumenti per accedere alle informazioni.

4.3.3 Aggregazioni

ArchivioLibri – Libro

L'archivio conserva l'insieme dei libri registrati, ma i libri vengono creati in modo indipendente tramite la loro classe. L'archivio si limita a organizzarli e a permetterne la consultazione.

ArchivioUtenti – Utente

Gli utenti non dipendono dall'archivio per la loro esistenza: quest'ultimo li raccoglie e li rende accessibili, ma non ne controlla il ciclo di vita. Per questo motivo la relazione è modellata come aggregazione.

ArchivioPrestitiAttivi – Prestito attivo

Anche i prestiti attivi sono entità autonome: vengono creati all'interno della logica di business e successivamente inseriti nell'archivio. L'archivio funge da contenitore dinamico, che riflette la situazione corrente dei prestiti attivi.

ArchivioCronologiaPrestiti – Prestito chiuso

L'archivio della cronologia gestisce i prestiti già conclusi, ma non ne determina la creazione né la distruzione. I prestiti, dopo essere stati chiusi, vengono semplicemente trasferiti nell'archivio storico, che ha il ruolo di conservarli e renderli consultabili.

4.4 Analisi della coesione e dell'accoppiamento

Questa sezione analizza la qualità strutturale del modello secondo i principi dell'ingegneria del software, in particolare:

- Tipologia di coesione (quanto una classe è focalizzata su un singolo compito)
- Tipologia di accoppiamento (quanto una classe dipende da altre)

4.4.1 Coesione

Nel sistema è stata ricercata una coesione alta in tutte le componenti principali, condizione essenziale per ottenere classi semplici da comprendere e modificare.

Classi di dominio (Libro, Utente, Prestito): coesione funzionale

Le classi di dominio presentano coesione funzionale, la forma più forte, perché ogni classe realizza un singolo compito ben definito:

- *Libro* gestisce esclusivamente i dati relativi a un volume.
- *Utente* conserva le informazioni anagrafiche e la lista dei prestiti attivi.

- *Prestito* rappresenta un'operazione di prestito con le sue date e il suo stato.

Ogni classe incapsula un concetto del dominio senza includere operazioni non attinenti.

Archivi (ArchivioLibri, ArchivioUtenti, ArchivioPrestitiAttivi, ArchivioCronologiaPrestiti): Coesione Comunicazionale

Gli archivi hanno coesione comunicazionale, perché tutte le operazioni al loro interno lavorano sugli stessi dati:

- collezione di libri,
- insieme di utenti,
- prestiti attivi o storicizzati.

Ricerca, aggiunta, rimozione e salvataggio su file agiscono tutte sulla stessa struttura dati. Non contengono logiche applicative.

Service (LibriService, UtentiService, PrestitiService): Coesione Sequenziale

Nei *Service* è presente una coesione sequenziale, perché le operazioni che essi svolgono devono essere eseguite in un ordine ben preciso, dove l'output di ciascun passo diventa l'input del successivo.

Ogni operazione dipende dal risultato della precedente, e l'intera procedura ha senso solo se eseguita in questa sequenza stabilita.

Controller (LibroViewController, UtentiViewController, PrestitiViewController, HomeViewController): Coesione Logica / Comunicazionale

I controller mostrano una coesione comunicazionale, perché tutte le operazioni lavorano sui dati che arrivano dalla UI e dai service (aggiornamento tabelle, lettura input, gestione feedback).

Presentano anche elementi di coesione logica, perché raccolgono diverse operazioni simili che riguardano l'interazione utente (clic, pulsanti, messaggi).

Non contengono logica applicativa e fungono da mediatori tra View e Service.

4.4.2 Accoppiamento

L'architettura mira a mantenere un accoppiamento debole. Nel sistema compaiono le seguenti tipologie.

Dominio → nessun accoppiamento forte

Le classi *Libro*, *Utente* e *Prestito* non dipendono dai Service, non dipendono dalla UI e non eseguono operazioni di I/O. L'unico accoppiamento presente è quello necessario tra le classi del dominio stesso (ad esempio *Prestito* è associato a *Utente* e *Libro*), sotto forma di

normale accoppiamento per dati. In questo modo il dominio rimane indipendente dagli altri livelli dell'applicazione e può essere riutilizzato o esteso senza modificare la logica di presentazione o di persistenza.

Archivi ↔ Dominio: Accoppiamento per Dati

Tra archivi e dominio avviene accoppiamento per dati:

- gli archivi ricevono o restituiscono solo le informazioni necessarie (ISBN, matricola, id prestito);
- non scambiano strutture o oggetti complessi inutili;
- non accedono direttamente agli attributi privati.

Service ↔ Archivi: Accoppiamento per Dati

I service si appoggiano agli archivi, ma:

- non ne controllano la struttura interna,
- usano solo i metodi pubblici,
- ricevono e passano solo i dati necessari.

Questo corrisponde a un accoppiamento per dati.

Controller ↔ Service: Accoppiamento per Controllo

I controller:

- delegano ai service le operazioni logiche,
- non alterano il loro comportamento interno,
- reagiscono ai valori restituiti o alle eccezioni.

Questo rappresenta un accoppiamento per controllo.

View ↔ Controller: Accoppiamento per dati

La View comunica esclusivamente con i controller e non ha alcuna dipendenza diretta da service, archivi o logica di business. Lo scambio avviene solo tramite i dati necessari per aggiornare l'interfaccia o per inoltrare l'input dell'utente, configurando quindi un accoppiamento per dati.

5. Modello dinamico

5.1 Introduzione ai diagrammi di sequenza

I diagrammi di sequenza rappresentano uno strumento fondamentale per comprendere la dinamica del sistema. Essi mostrano l'ordine temporale con cui i messaggi vengono scambiati tra gli oggetti durante l'esecuzione di un caso d'uso specifico.

Nel contesto dell'applicazione biblioteca, i diagrammi di sequenza evidenziano:

- la separazione netta tra controller, service e archivi, confermando l'adozione del pattern MVC;
- il flusso delle operazioni, dalla ricezione dell'input utente alla persistenza dei dati;
- i punti di validazione e controllo, dove vengono applicate le regole di dominio;
- la gestione delle eccezioni e dei flussi alternativi.

Ogni diagramma è accompagnato da una descrizione testuale che spiega il flusso principale e le possibili deviazioni, fornendo una visione completa del comportamento del sistema.

5.2 Sequenza: Registrazione di un libro

La registrazione di un nuovo libro nel catalogo rappresenta una delle operazioni fondamentali del sistema. Come si constata in Figura 6, il processo coinvolge il LibroViewController, che gestisce l'interfaccia utente, il LibriService, che applica la logica di business, e l'ArchivioLibri, che si occupa della persistenza.

5.2.1 Flusso Principale

1. Il bibliotecario accede alla sezione “Libri” e visualizza la schermata apposita.
2. Il bibliotecario compila i campi obbligatori: titolo, lista degli autori, anno di pubblicazione, ISBN e numero di copie disponibili.
3. Dopo aver premuto il pulsante “Aggiungi nuovo libro”, il LibroViewController intercetta l'evento associato al metodo onAggiungiLibro().
4. Il service effettua una validazione dei dati inseriti, verificando che:
 - tutti i campi obbligatori siano compilati;
 - l'anno sia un valore numerico valido;
 - l'ISBN rispetti il formato previsto;
 - il numero di copie sia un valore intero non negativo.
5. Se la validazione ha esito positivo il LibriService effettua un controllo di unicità interrogando l'ArchivioLibri tramite il metodo ricercaISBN(ISBN).
6. L'ArchivioLibri esegue una ricerca nella propria collezione interna e restituisce il risultato.
7. Se l'ISBN non è presente (la ricerca restituisce NULL), il LibriService crea un'istanza della classe Libro con i dati forniti.
8. Il LibriService procede con l'inserimento invocando aggiungiLibro(libro) sull'ArchivioLibri.
9. L'ArchivioLibri inserisce il nuovo libro nella struttura e conferma l'operazione.

10. Il controllo ritorna al LibroViewController che mostra un messaggio di conferma all'utente.
11. Il nuovo libro rimane memorizzato nell'archivio in memoria e sarà salvato permanentemente su file al momento della chiusura dell'applicazione, quando viene invocato automaticamente il metodo `scriviSuFile()`.

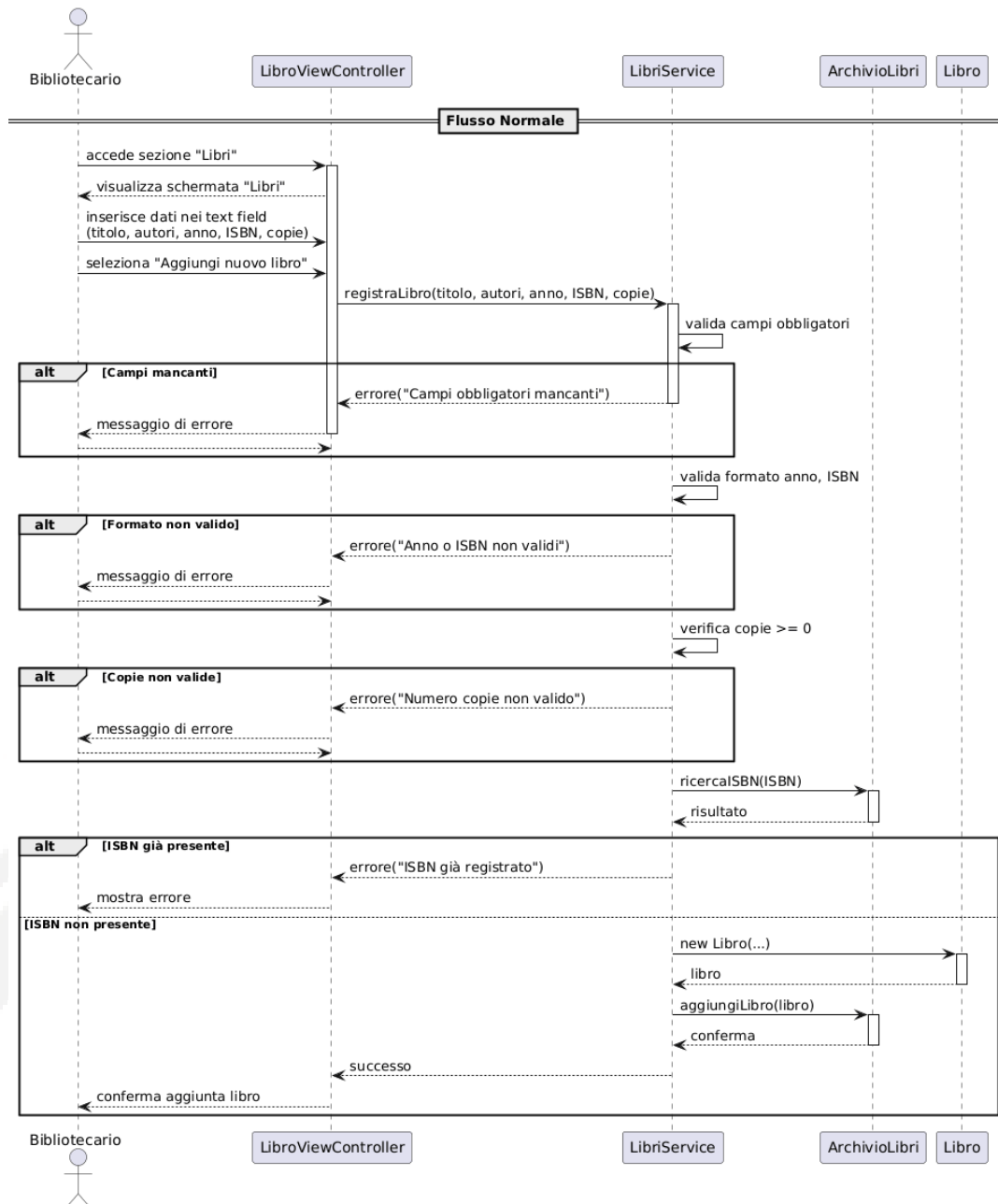


Figura 6 - Diagramma di sequenza: registrazione di un libro

5.2.2 Gestione dei flussi alternativi

Dati mancanti o non validi

Se la validazione locale dei dati fallisce (presenza di campi vuoti, formato anno o ISBN errato, numero di copie negativo), il controller interrompe immediatamente il flusso e mostra un messaggio di errore all'utente. L'operazione non viene propagata al service, evitando elaborazioni inutili. Il bibliotecario può correggere i dati e riprovare l'inserimento.

ISBN già presente in archivio

Se l'ISBN risulta già registrato, il metodo `ricercaISBN()` restituisce un oggetto `Libro` invece di `NULL`. Il `LibriService` rileva la duplicazione e solleva un'eccezione di tipo `duplicatoException`. Il controller intercetta l'eccezione e mostra il messaggio "ISBN già registrato" all'utente. L'esecuzione riprende permettendo al bibliotecario di modificare l'ISBN o annullare l'operazione.

5.3 Sequenza: Registrazione di un prestito

La registrazione di un prestito rappresentata in Figura 7 è l'operazione più complessa del sistema, poiché coinvolge tre archivi distinti (libri, utenti e prestiti) e richiede numerose validazioni per garantire la coerenza del dominio. Il processo mette in relazione un utente con un libro, creando un'entità `Prestito` che attraverserà diverse fasi del proprio ciclo di vita.

5.3.1 Flusso principale

1. Il bibliotecario accede alla sezione "Prestiti" e visualizza l'interfaccia dedicata.
2. Il sistema richiede l'inserimento della matricola dell'utente. Il bibliotecario inserisce il valore e avvia la ricerca.
3. Il `PrestitiViewController` delega la ricerca al `UtentiService` invocando il metodo `cercaUtentePerMatricola(matricola)`.
4. L'`UtentiService` valida che la matricola sia un numero positivo, quindi interroga l'`ArchivioUtenti` tramite `ricercaMatricola(matricola)`.
5. L'`ArchivioUtenti` esegue la ricerca e, se l'utente esiste, restituisce l'oggetto `Utente` corrispondente.
6. I dati dell'utente (nome, cognome, email) vengono mostrati nell'interfaccia per conferma visiva.
7. Analogamente, il bibliotecario inserisce l'ISBN del libro da prestare e avvia la ricerca.
8. Il controller delega al `LibriService`, che interroga l'`ArchivioLibri` tramite `ricercaISBN(ISBN)`.
9. Se il libro esiste, i suoi dati (titolo, autore, copie disponibili) vengono visualizzati.
10. A questo punto, il service verifica che l'utente selezionato non abbia già tre prestiti attivi. Per farlo, interroga l'`ArchivioPrestitiAttivi` con `getPrestitiAttivi()` e conta i prestiti associati all'utente corrente.
11. Se l'utente ha meno di tre prestiti attivi, il bibliotecario seleziona "Registra prestito" e inserisce la data prevista di restituzione nel formato GG/MM/AAAA.
12. Il controller valida il formato della data e verifica che non sia precedente alla data odierna.
13. Il service controlla che il libro abbia almeno una copia disponibile invocando `getCopieDisponibili()` sull'oggetto `Libro`.

14. Se tutte le verifiche hanno esito positivo, Il PrestitiService crea una nuova istanza di Prestito utilizzando il costruttore che accetta utente, libro, data inizio e data fine. Il costruttore genera automaticamente un ID univoco incrementando un contatore statico e imposta lo stato iniziale a ATTIVO.
15. Il service registra il prestito in entrambi gli archivi:
 - invoce aggiungiPrestitoAttivo(prestito) sull'ArchivioPrestitiAttivi per la gestione operativa quotidiana.
 - invoce aggiunguPrestitoCronologia(prestito) sull'ArchivioCronologiaPrestiti per mantenere lo storico completo.
16. Il service aggiorna la disponibilità del libro invocando decrementaCopie() sull'oggetto Libro.
17. Il service conferma il successo dell'operazione al controller.
18. Il PrestitiViewController mostra un messaggio di conferma all'utente: "Prestito registrato con successo".
19. Il nuovo prestito rimane memorizzato in memoria e sarà salvato su file alla chiusura dell'applicazione.





Figura 7 - Diagramma di sequenza: registrazione di un prestito

5.3.2 Gestione dei flussi alternativi

Utente non trovato

Se la matricola inserita non corrisponde ad alcun utente nell'archivio, il metodo `ricercaMatricola` restituisce NULL.

L'`UtentiService` solleva un'eccezione `UtenteNonTrovatoException`, il controller intercetta l'eccezione e mostra il messaggio "Nessun utente trovato" all'utente.

L'operazione termina senza registrazione del prestito e il bibliotecario può riprovare con una matricola diversa.

Libro non trovato

Analogamente, se l'ISBN non esiste nell'archivio, viene sollevata un'eccezione di tipo `LibroNonTrovatoException` e mostrato il messaggio "Nessun libro trovato". L'operazione termina senza ulteriori elaborazioni.

Limite massimo prestiti raggiunto

Se il conteggio dei prestiti attivi dell'utente raggiunge il valore 3, viene sollevata l'eccezione `LimitePrestitoSuperatoException`, il controller mostra il messaggio "Limite massimo di prestiti raggiunto" e impedisce la prosecuzione. Questa verifica avviene prima di richiedere la data di restituzione, evitando inserimenti inutili.

Nessuna copia disponibile

Se il metodo `getCopieDisponibili()` restituisce 0 il controller intercetta l'eccezione `LibroNonDisponibileException` e mostra "Nessuna copia disponibile" e l'operazione non viene registrata.

Questo controllo è fondamentale per mantenere la coerenza tra il catalogo e i prestiti attivi.

Data di restituzione non valida

Se il formato della data non rispetta lo schema GG/MM/AAAA o se la data è precedente a quella odierna, viene mostrato un messaggio di errore appropriato. Il bibliotecario può correggere la data e riprovare.



5.4 Sequenza: Restituzione di un prestito

La restituzione di un libro in Figura 8 conclude il ciclo di vita di un prestito attivo, aggiornando il suo stato e ripristinando la disponibilità del volume nel catalogo. Questa operazione richiede la collaborazione tra più componenti per garantire che tutte le informazioni rimangano allineate.

5.4.1 Flusso principale

1. Il bibliotecario accede alla sezione "Prestiti" e seleziona l'opzione "Prestiti attivi" oppure utilizza la funzione "Ricerca prestito".
2. Se viene selezionata la visualizzazione completa, il `PrestitiViewController` invoca `visualizzaPrestitiAttivi()` sul `PrestitiService`.
3. Il `PrestitiService` interroga l'`ArchivioPrestitiAttivi` con il metodo `getPrestitiAttivi()`.
4. L'archivio restituisce la lista completa dei prestiti in corso, che viene mostrata nella tabella dell'interfaccia.
5. Se invece viene utilizzata la funzione di ricerca specifica (ad esempio per utente e libro), il sistema mostra l'elenco dei prestiti attivi con tutte le informazioni rilevanti: ID del prestito, nome e cognome dell'utente, matricola, titolo del libro, ISBN, data di inizio e data prevista di restituzione.
6. Il bibliotecario seleziona dalla tabella il prestito relativo al libro restituito.
7. Il bibliotecario conferma l'operazione cliccando su "Conferma Restituzione".
8. Il sistema richiede l'inserimento della data effettiva di restituzione.
9. Il Service valida che la data inserita rispetti il formato GG/MM/AAAA.
10. Se la validazione ha esito positivo, il controller delega la chiusura del prestito al `PrestitiService` invocando `eliminaPrestitoAttivo(prestito)`.
11. Il `PrestitiService` recupera la data prevista di restituzione dall'oggetto `Prestito` tramite il metodo `getDataFine()`.
12. Il service confronta la data effettiva di restituzione con la data prevista per determinare se si è verificato un ritardo.
13. Se la data effettiva è successiva alla data prevista, il service aggiorna lo stato del prestito invocando `setStato(RITARDO)` sull'oggetto `Prestito`. Altrimenti lo stato viene impostato a CHIUSO.
14. Il `PrestitiService` rimuove il prestito dall'`ArchivioPrestitiAttivi` invocando `rimuoviPrestitoAttivo(prestito)`.
15. Il service recupera il libro associato al prestito tramite `getLibro()` e invoca `incrementaCopie()` per ripristinare la disponibilità nel catalogo.
16. Il service conferma il successo dell'operazione al controller.
17. Il `PrestitiViewController` mostra il messaggio "Restituzione registrata con successo" all'utente.

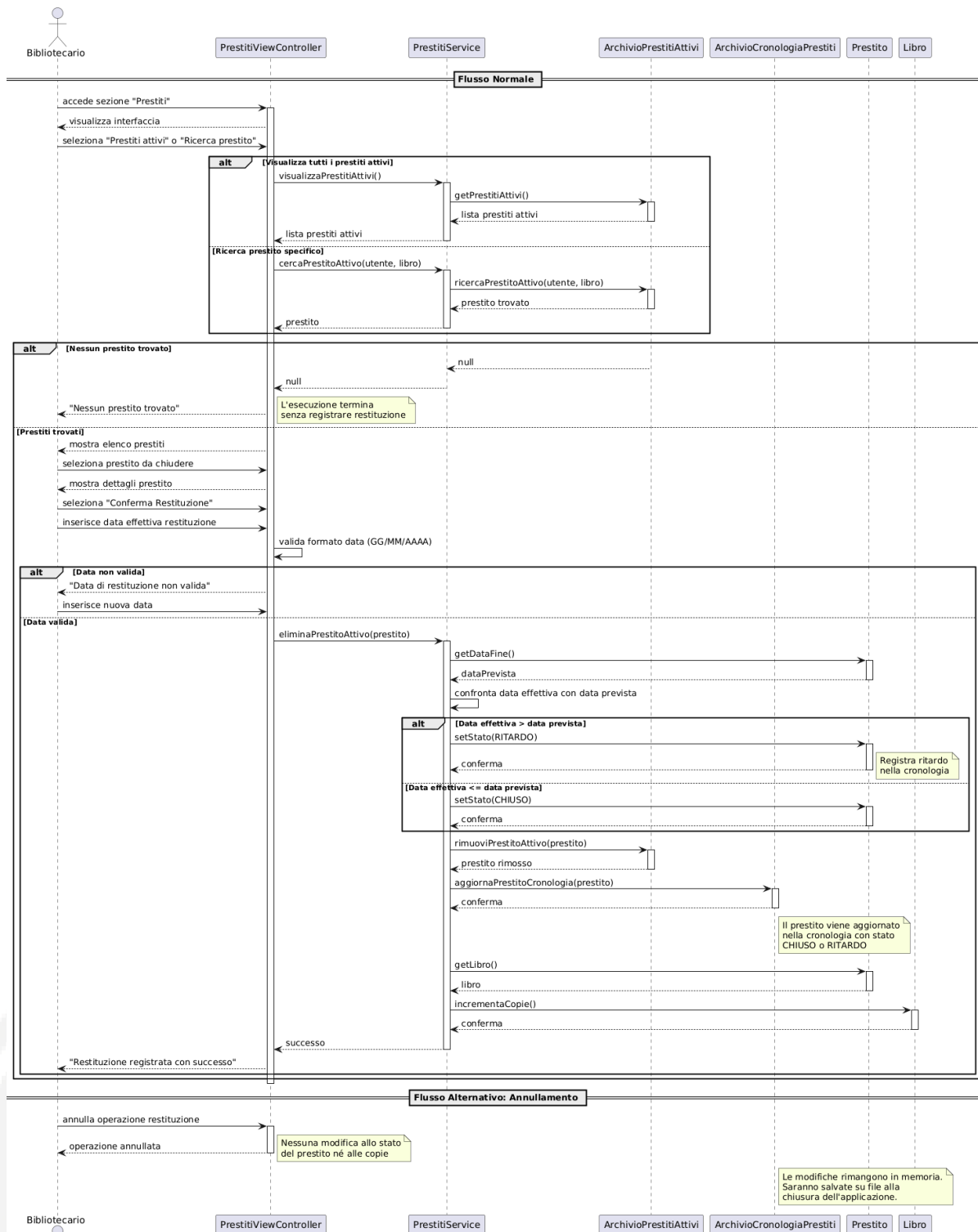


Figura 8 - Diagramma di sequenza: restituzione di un prestito

5.4.2 Gestione dei flussi alternativi

Nessun prestito trovato

Se non esistono prestiti attivi corrispondenti ai criteri di ricerca, o se l'elenco dei prestiti attivi è vuoto, il sistema solleva l'eccezione `PrestitoNonTrovatoException` e mostra il messaggio "Nessun prestito trovato". L'operazione termina immediatamente senza apportare modifiche agli archivi.

Data di restituzione non valida

Se il formato della data inserita non rispetta lo schema GG/MM/AAAA, il controller invoca l'eccezione `ValidazioneException` e mostra un messaggio di errore del tipo "Data di restituzione non valida". L'esecuzione ritorna al punto in cui viene richiesta la data, permettendo al bibliotecario di correggerla e riprovare.

Annullamento dell'operazione

Se il bibliotecario decide di annullare l'operazione prima della conferma finale (ad esempio chiudendo la finestra di dialogo o premendo un pulsante "Annulla"), il sistema non modifica né lo stato del prestito né il numero di copie disponibili. L'esecuzione termina senza impatto sugli archivi.

5.5 Diagramma Macchina a Stati Finiti

Il diagramma a stati finiti in Figura 9 descrive l'evoluzione dell'oggetto `Prestito`, che può assumere tre stati distinti:

- **Attivo:** Stato iniziale. Si attiva alla conferma della registrazione se le precondizioni sono soddisfatte (copie disponibili > 0, limite prestiti utente non superato). Le copie del libro vengono decrementate.
- **In Ritardo:** Transizione automatica. Si verifica se la data odierna supera la data di scadenza prevista senza che il libro sia stato restituito. L'anomalia viene evidenziata nell'interfaccia.
- **Chiuso:** Stato finale. Raggiunto al momento della restituzione del libro (sia da stato Attivo che In Ritardo). Le copie disponibili vengono incrementate e il prestito viene archiviato nello storico.

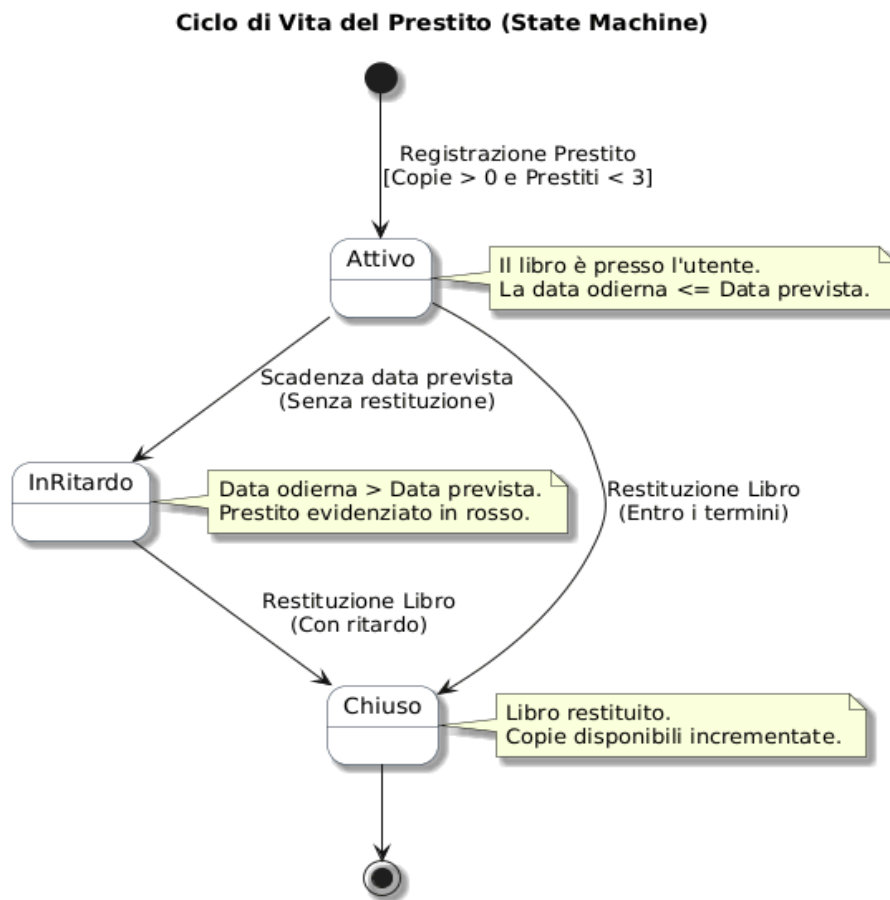


Figura 9 - Diagramma di sequenza: registrazione di un prestito

5.6 Descrizione del flusso dell'applicazione

Il diagramma in Figura 10 mostra il flusso principale dell'applicazione dalla fase di avvio fino alla chiusura. Dopo l'apertura del programma, l'utente viene indirizzato alla schermata *Home*, dalla quale può selezionare una delle tre sezioni disponibili: Libri, Utenti o Prestiti. Una volta scelta la sezione, l'applicazione visualizza l'interfaccia corrispondente e l'utente può eseguire diverse operazioni, come aggiunta, ricerca, modifica o eliminazione dei dati.

Il flusso prevede due controlli ricorrenti: la possibilità di uscire dall'applicazione e la possibilità di cambiare sezione. Se l'utente seleziona "Esci", il programma termina; se invece decide di spostarsi a un altro modulo, il flusso ritorna alla selezione della schermata da visualizzare. In assenza di queste azioni, l'utente rimane nella stessa schermata e può continuare a operare. Questo modello descrive quindi il comportamento interattivo e ciclico dell'interfaccia, garantendo continuità e coerenza nell'esperienza d'uso dell'applicazione.

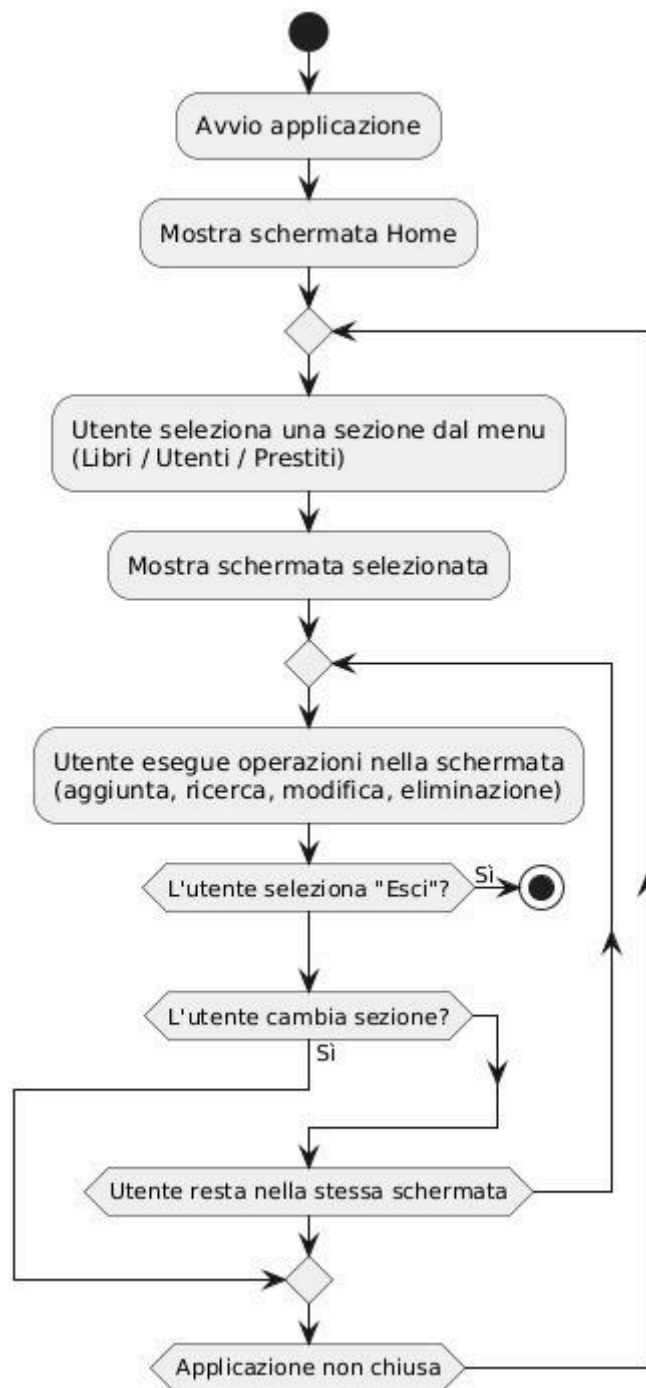


Figura 10 - Diagramma delle attività: gestione

6. Design dell'interfaccia utente

6.1 Linee guida adottate

L'interfaccia utente è stata progettata seguendo criteri di semplicità, coerenza e massima leggibilità. Ogni schermata utilizza lo stesso schema grafico, così da permettere al bibliotecario di riconoscere rapidamente funzioni e pattern di interazione.

La distribuzione dei contenuti privilegia un approccio “a pannelli”: a sinistra si trovano i campi di inserimento e gli strumenti per attivare le operazioni, mentre la parte destra è riservata alla visualizzazione dei dati risultanti da ricerche o aggiornamenti.

Sono stati inoltre adottati:

- layout chiari e non affollati, per ridurre il carico cognitivo;
- colori moderati e coerenti in tutta l'applicazione, con tinte più accese solo per pulsanti rilevanti;
- componenti grafici standard per garantire familiarità;
- messaggi di feedback immediati, così da informare l'utente su esiti, errori o conferme.

6.2 Navigazione tra le schermate

Il sistema utilizza una barra di navigazione orizzontale, sempre presente nella parte superiore dell'interfaccia.

Home Libri Utenti Prestiti

Da questa barra l'utente può accedere direttamente alle quattro sezioni principali: *Home*, *Libri*, *Utenti* e *Prestiti*. Questa scelta progettuale garantisce una navigazione fluida e immediata, riducendo il numero di passaggi necessari per raggiungere una data funzione. La barra rimane invariata in tutte le schermate, fornendo continuità e facilitando l'orientamento dell'utente.

6.3 Mock-up delle schermate

L'applicazione organizza la sua interfaccia in quattro sezioni principali — *Home*, *Libri*, *Utenti* e *Prestiti* — ognuna con una funzione specifica. Questa suddivisione permette di isolare attività diverse e di mantenere ogni area focalizzata sulle operazioni previste dal modulo corrispondente.

6.3.1 Home

La Home (Figura 11) rappresenta il punto di accoglienza dell'utente. È una schermata essenziale, incentrata sul menù di navigazione che consente di raggiungere rapidamente tutte le altre sezioni.

Gli elementi grafici sono volutamente discreti: la parte centrale mostra un'immagine

istituzionale, mentre la barra superiore dà accesso immediato ai moduli *Libri*, *Utenti* e *Prestiti*. La Home non contiene campi operativi, ma funge da snodo principale dell'applicazione.



Figura 11 - Home

6.3.2 Schermata Libri

La schermata Libri in Figura 12 consente al bibliotecario di gestire completamente l'archivio dei volumi.

- **Pannello di sinistra**
Qui sono presenti i campi per inserire o ricercare i dati dei libri. Il bottone *Visualizza Tutti i Libri* consente di ottenere l'elenco completo dell'archivio. I pulsanti presenti sotto i campi permettono di eseguire le diverse operazioni, ma risultano attivi solo quando i campi necessari all'operazione selezionata sono stati compilati. In questo modo l'interfaccia previene errori di mancato inserimento e guida il bibliotecario verso un utilizzo corretto delle funzioni disponibili.
- **Pannello di destra**
È dedicato alla visualizzazione dei risultati. La tabella mostra i libri trovati e permette la selezione di un elemento per modificarne i dati o rimuovere l'intero elemento cliccando in seguito sul bottone di eliminazione.

La schermata mantiene la stessa logica visiva adottata negli altri moduli, riducendo i tempi di adattamento del bibliotecario e rendendo immediato il passaggio da una sezione all'altra.

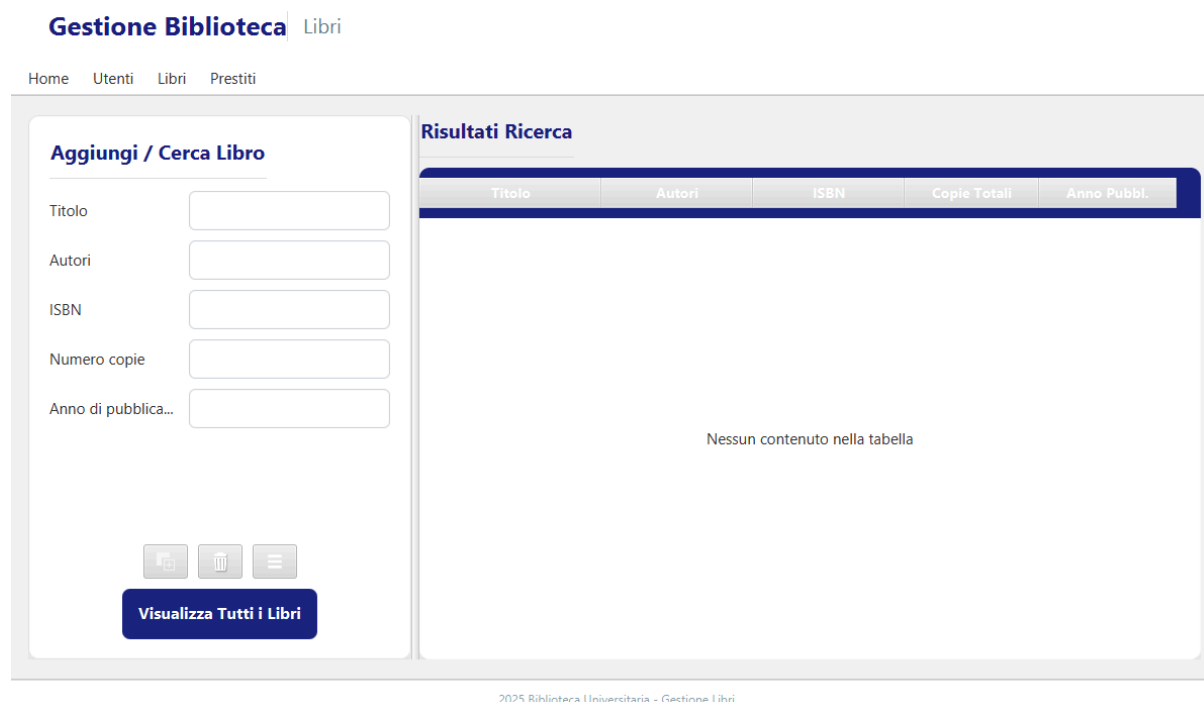


Figura 12 - Schermata Libri

6.3.3 Schermata Utenti

La schermata Utenti in Figura 13 consente al bibliotecario di gestire completamente l'archivio degli utenti registrati.

- **Pannello di sinistra**
Qui sono presenti i campi per inserire o ricercare i dati dell'utente. Il bottone *Visualizza Utenti* consente di ottenere l'elenco completo dell'archivio. I pulsanti presenti sotto i campi permettono di eseguire le diverse operazioni, ma risultano attivi solo quando i campi necessari all'operazione selezionata sono stati compilati. In questo modo l'interfaccia previene errori di mancati inserimento e guida il bibliotecario verso un utilizzo corretto delle funzioni disponibili
- **Pannello di destra**
È dedicato alla visualizzazione dei risultati. La tabella mostra gli utenti trovati e permette la selezione di un elemento per modificarne i dati o rimuovere l'intero elemento cliccando in seguito sul bottone di eliminazione.

La schermata mantiene la stessa logica visiva adottata negli altri moduli, riducendo i tempi di adattamento del bibliotecario e rendendo immediato il passaggio da una sezione all'altra.

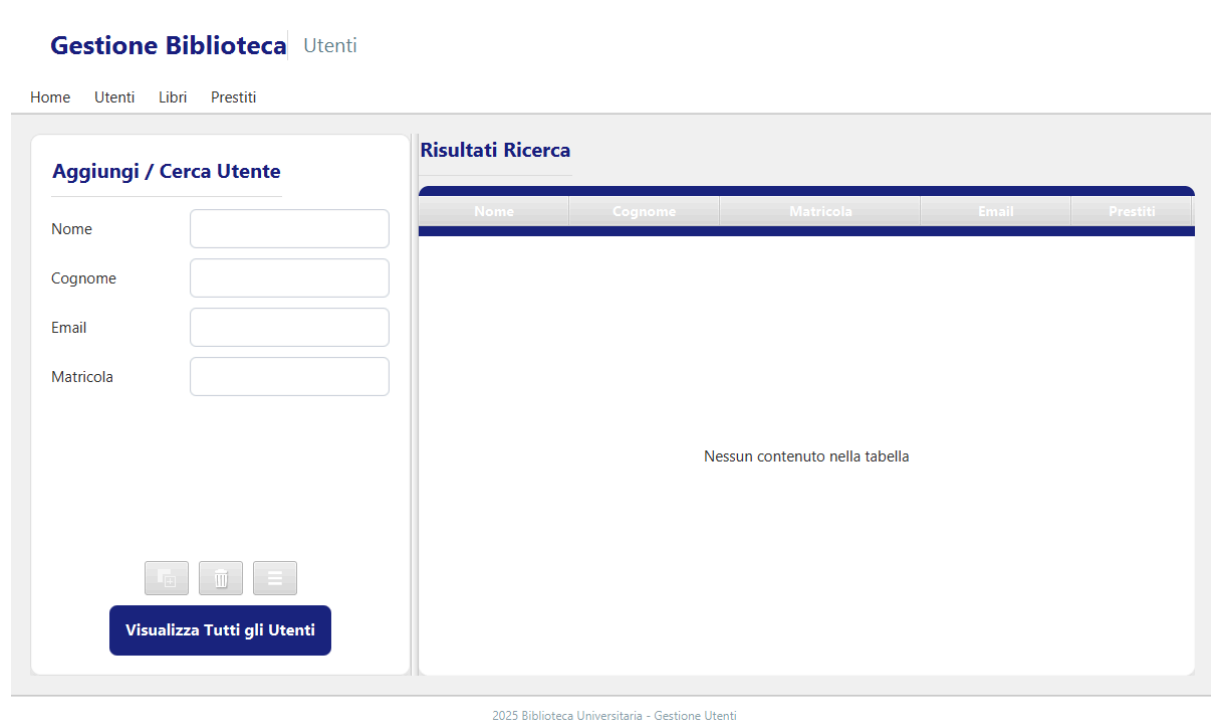


Figura 13 - Schermata Utenti

6.3.4 Schermata Prestiti

La sezione Prestiti (Figura 14) integra operazioni più articolate, poiché coinvolge utenti, libri e gestione dello stato dei prestiti.

- **Registrazione di un nuovo prestito:**
L'utente inserisce ISBN del libro, matricola dello studente e data di restituzione. Premendo il pulsante dedicato, il sistema verifica la disponibilità del libro, l'esistenza dell'utente e la validità della data, registrando il prestito in caso positivo.
- **Restituzione di un prestito**
In fase di restituzione, l'utente inserisce ISBN, matricola e data di restituzione e conferma l'operazione tramite il comando Restituzione. Il sistema valida i dati e aggiorna automaticamente lo stato del prestito da Attivo o In Ritardo a Chiuso.
- **Visualizzazione**
La tabella inferiore mostra, tramite gli appositi pulsanti *Visualizza Prestiti* e *Cronologia* rispettivamente i prestiti attivi e la cronologia dei prestiti conclusi.

In questa schermata i prestiti in ritardo sono evidenziati graficamente in rosso, rendendoli immediatamente riconoscibili. I pulsanti presenti permettono di eseguire le diverse operazioni, ma risultano attivi solo quando i campi necessari all'operazione selezionata sono stati compilati. In questo modo l'interfaccia previene errori di mancata inserimento e guida il bibliotecario verso un utilizzo corretto delle funzioni disponibili.

Gestione Biblioteca | Prestiti

Home Utenti Libri Prestiti

Ricerca Libro

ISBN

Ricerca Utente

Matricola

Inserisci

Data di Restituzione

Restituzione

Prestiti Registrati

ID	Nome	Cognome	Matricola	Email	Titolo	Autori	ISBN	Copie	Data Inizio	Data Fine	Stato
Nessun contenuto nella tabella											

Visualizza Prestiti **Cronologia**

2025 Biblioteca Universitaria - Gestione Prestiti

Figura 14 - Schermata Prestiti

6.4 Gestione dei messaggi

L'applicazione comunica costantemente con l'utente tramite finestre di dialogo (Alert) che forniscono un feedback chiaro e tempestivo.

Si distinguono tre categorie principali:

- **Messaggi di errore (Figura 15)**
Segnalano dati non validi, ricerche senza risultato o operazioni non consentite, bloccano temporaneamente il flusso.
- **Conferme di operazioni**
Utilizzate per azioni sensibili come eliminazioni o modifiche di dati già presenti nell'archivio.
- **Messaggi informativi**
Visualizzati quando un'operazione va a buon fine, come la chiusura di un prestito.

La gestione uniforme di questi messaggi assicura chiarezza ed evita comportamenti ambigui all'interno dell'applicazione.

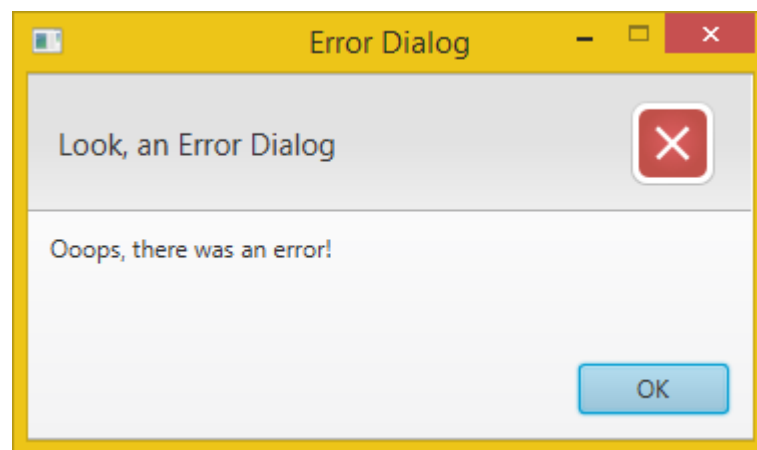


Figura 15 - Gestione dei messaggi

